

TD7- Liste doublement chaînées (Gestion des Polynômes)

EXERCICE 1

On souhaite écrire des fonctions permettant d'effectuer diverses opérations sur des polynômes d'une variable réelle. On rappelle qu'un polynôme est constitué d'une somme de **monômes non nuls**. Chaque monôme est représenté par une structure composée de deux membres : degré (un entier) et coefficient (un réel).

Les polynômes seront représentés par une **liste doublement chaînée** de monômes non nuls (i.e : chaque cellule de la liste représente un monôme de coefficients non nuls) et **classés dans l'ordre croissant selon le degré**.

La liste doublement chaînée représentant le polynôme est définie comme suit :

typedef struct{ double coef ; int degre ; } Monome ;	typedef struct cellule{ Monome monome ; struct cellule* suivant ; struct cellule* precedent ; }*Polynome ;
---	--

1) Montrer graphiquement la représentation de la structure du polynôme suivant :

$$P(x) = 13 - 2.5x^2 + x^5 + 2x^8 - 7x^{13}$$

2) Ecrire les différentes fonctions de base suivantes qui sont liées à la structure d'un Monôme :

- **degre** : Monôme → Entier // Retourne le degré d'un monôme.
- **coef** : Monôme → Réel // Retourne le coefficient d'un monôme

3) Ecrire les différentes fonctions de base suivantes qui sont liées à la structure d'un polynôme (=liste). Soit P un polynôme donné :

- **monôme** : Polynôme → Monôme // Retourne le monôme associé au polynôme donné.
- **suivant** : Polynôme → Polynôme // Retourne l'adresse du monôme qui suit le monôme pointé par un polynôme donné.
- **precédent** : Polynôme → Polynôme // Retourne l'adresse du monôme qui précède le monôme pointé par un polynôme donné.

4) Ecrire les fonctions associées aux opérations suivantes :

- **initP** : {} → Polynôme // Crée un polynôme vide.
- **estVideP** : Polynôme → Booléen // teste si un polynôme donné est vide ou non.
- **inserMonPoly** : Monôme x Polynôme → Polynôme // insertion d'un monôme donné dans un polynôme p donné. Le polynôme p est modifié.
- **rechMonPoly** : Monôme x Polynôme → Polynôme // retourne l'adresse du monôme s'il appartient au polynôme NULL sinon.
- **supplMonPoly** : Monôme x Polynôme → Polynôme // suppression d'un monôme donné dans un polynôme p donné. Le polynôme p est modifié.
- **delPolynôme** : Polynôme → // détruit un polynôme donné.

5) Applications sur les polynômes :

- **maxCoef** : Polynôme → Réel // retourne le plus grand coefficient du polynôme.
- **somCoef** : Polynôme → Réel // retourne la somme des coefficients d'un polynôme
- **generPoly** : Entier x Tableau → Polynôme // génère un polynôme à partir d'un tableau composé de n coefficients tel que tab[i] est le coefficient de xⁱ.
- **affichPoly** : Polynôme → {} // affiche les degrés et les coefficients des différents monômes.
- **derivPoly** : Polynôme → Polynôme // détermine la dérivée d'un polynôme p donné. Le polynôme p ne doit pas être modifié.
- **somPoly** : Polynôme x Polynôme → Polynôme // détermine la somme deux polynômes donnés p et q. Les polynômes p et q ne doivent pas être modifiés.

6) Ecrire un programme principal avec un **menu** pour tester les fonctions définies ci-dessus.

EXERCICE 2

Réécrire les structures de données ainsi que les algorithmes précédents en supposant que la liste doublement chaînée est définie par **deux adresses** : la première adresse pour indiquer la tête de la liste et la seconde adresse pour indiquer la queue de la liste.